EXPERIMENT NO. 6

Implement Constructors, this, static Members and Encapsulation

PART B – TO BE COMPLETED BY STUDENTS

Roll No. C-60

Name Avdhut Parish Madhavi

Class / Division SE-C

Batch C2

Date of Experiment _____

Date of Submission _____

Grade _____

B.1 Document Created by the Student

Change the class to include one setter method and demonstrate controlled modification of an attribute.

Modification: A setAge() method is added to modify the patient's age while keeping the data member private.

B.2 Student Program / Output

Paste/write the final program and attach or reproduce the observed output below.

Java Program

```
class PatientRecord {
    private int id;
    private String name;
    private int age;
    private static int count = 0;

    // Parameterized constructor
    PatientRecord(int id, String name, int age) {
        this.id = id;
        this.name = name;
        this.age = age;
        count++;
    }

    // Getter methods
    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }
```

```
    public int getAge() {
        return age;
    }

    // Setter method
    public void setAge(int age) {
        if (age > 0) {
            this.age = age;
        }
    }

    // Static method
    public static int getCount() {
        return count;
    }

    public void display() {
        System.out.println(id + " " + name + " " + age);
    }
}

public class EncapsulationDemo {
    public static void main(String[] args) {
        PatientRecord p1 =
            new PatientRecord(101, "Aarav", 21);

        PatientRecord p2 =
            new PatientRecord(102, "Meera", 22);

        System.out.println("Before modification:");
        p1.display();
        p2.display();

        // Modify age using setter
        p1.setAge(23);

        System.out.println("After modification:");
        p1.display();

        System.out.println("Patient objects: "
                           + PatientRecord.getCount());
    }
```

}

Output:

```
Before modification:
101 Aarav 21
102 Meera 22
After modification:
101 Aarav 23
Patient objects: 2
```

B.3 Observations and Learning

- Why is encapsulation useful?
  Encapsulation protects data by restricting direct access to class variables. Private variables can be accessed or modified through public getter and setter methods.

- What does this refer to?
  The this keyword refers to the current object. It is used to distinguish instance variables from constructor or method parameters with the same names.
  Example:
  this.age = age;

- Why is count declared static?

  The count variable is declared static because it is shared by all objects of the PatientRecord class. It keeps track of the total number of objects created.

Student observation / learning:

The program demonstrates parameterized constructors, private data members, getter and setter methods, the this keyword, and static members. The setter method modifies the patient's age only when the new value is greater than zero. The static counter displays the total number of patient objects created.

B.4 Conclusion

Secure access control in medical software is listed as a self-learning context in the syllabus.

Student conclusion:

The experiment successfully demonstrated constructors, the this keyword, static members, and encapsulation in Java. Private data members and setter methods provided controlled access to object data, while the static counter tracked the number of objects created.

B.5 Questions of Curiosity

1. Why is encapsulation useful?
   Encapsulation protects an object's data from direct external access and allows controlled modification through methods.

2. What does this refer to?
   this refers to the current object of a class. It helps distinguish instance variables from parameters having the same names.

3. Why are static members associated with the class?

   Static members belong to the class rather than individual objects. They are shared among all instances, so a single copy of a static variable can maintain common information such as an object count.